



Sortable LMS API comparison: Canvas vs Blackboard vs Moodle vs D2L Brightspace — rate limits, OAuth scopes, and LTI 1.3 endpoints

Computer

Gradescope vs Copyleaks vs Turnitin AI grading: compare accuracy, false-positive rates, and plagiarism workflows side-by-side

Computer

Compare API rate limits for all four LMS platforms

Show the full LTI 1.3 implementation matrix

Detail the QTI 3.0 migration steps and tools

The practical answer is: build Semester around standards, not any LMS vendor’s proprietary API behavior. Use LTI 1.3/LTI Advantage for launch, context, roster, and grade exchange; use each LMS’s API only for explicit, customer-approved workflows; and treat rate limits/webhooks as variable tenant-level operational constraints.

API rate-limit comparison

A fixed, comparable requests-per-minute number is **not publicly established across all four platforms**. Canvas and Brightspace publish meaningful throttling guidance; Blackboard and Moodle are more deployment/configuration dependent, so their limits must be confirmed in the customer’s actual tenant and contract.

LMS	Public API auth model	Published rate-limit behavior	Integration design implication	Confidence
Canvas	Developer Keys and scoped API/LTI configuration; OAuth-based API access	Dynamic throttling: calls have a cost against a replenishing quota; throttled calls return HTTP 429	Read quota/throttling headers, constrain concurrent calls, use jittered exponential backoff, persist idempotency keys, and reconcile state	High for mechanism; actual capacity varies
Blackboard Learn	OAuth 2.0 key/secret and application registration; API enabled per Learn instance	Public/current production limit is not consistently disclosed; older framework material described an early technical-preview limit, which should not be used as a present production benchmark	Confirm supported Learn deployment/version, endpoint access, quota, token behavior, and event options during tenant discovery; build generic 429/circuit-breaker handling	Medium/low until customer confirms
Moodle	Admin-enabled Web Services and tokens; OAuth 2 configuration also exists for external systems; plugin/hosting model varies	No universal core SaaS-style quota applies across all Moodle deployments; institution, host, reverse proxy, plugin, and custom controls can determine behavior	Treat each Moodle instance as unique; negotiate/load-test approved quotas; favor incremental sync, pagination, checkpointing, and reconciliation	Low as a universal number; high that variance is real

LMS	Public API auth model	Published rate-limit behavior	Integration design implication	Confidence
D2L Brightspace	Brightspace Developer Platform / REST-like API and application credentials	Token-bucket credit model; published FAQ says 50,000 API credits per minute, while individual calls can have variable dynamic cost; response headers provide remaining budget, cost, reset, and retry timing	Dynamically adapt worker concurrency to headers; cache reads; batch where supported; honor Retry-After; keep grade writes idempotent	High for mechanism; per-tenant operational policy still verify

Canvas documents dynamic throttling based on request cost and a replenishing quota, with HTTP 429 on throttled calls. [^1642] Brightspace documents a token-bucket model, variable request cost, rate-limit headers, Retry-After, and HTTP 429 behavior; its published FAQ identifies a 50,000-credit-per-minute bucket. [1638][1635] Blackboard's API framework confirms OAuth 2.0 app registration and instance-level enablement, but the older 10,000-requests-per-24-hours figure was explicitly a technical-preview setting and should not be treated as current production capacity. [^1648]

Required adaptive client behavior

Read response headers on every API call.
Track quota/credits by tenant, credential, endpoint family, and worker.
Use bounded concurrency rather than a fixed global parallelism level.
Retry only transient failures: 429, 408, selected 5xx, and network errors.
Honor Retry-After where present.
Use exponential backoff with jitter.
Use idempotency keys for writes where supported; otherwise use a local write ledger.
Pause/circuit-break on sustained failures.
Use durable queues and a dead-letter queue.
Run scheduled reconciliation because API/webhook delivery can be incomplete or delayed.
Expose sync lag, failed records, retry state, and repair actions in the Operations Console.

Full LTI 1.3 matrix

LTI 1.3 is the common security baseline: OpenID Connect login initiation, signed JWT launch messages, platform JWKS verification, issuer/audience/deployment validation, nonce/state protection, and OAuth 2 client-credentials access tokens for Advantage services.

Capability	Canvas	Blackboard Learn	Moodle	D2L Brightspace	Semester requirement
LTI 1.3 Core launch	Supported through Developer Key / external-tool configuration	Supported on eligible SaaS and hosted Learn deployments	Supported in current configured LTI platform/tool flows	Supported; validate configured tool/provider services	Required
OIDC login initiation	Tool configuration provides login-initiation and target-link setup	Tenant/admin configuration required	Supported security model	Tenant/admin configuration required	Validate issuer, client ID, state, nonce, redirect URI
JWT launch validation	Platform JWKS and claims validation	Platform JWKS and claims validation	Platform JWKS and claims validation	Platform JWKS and claims validation	Required: signature, issuer, audience, deployment, nonce, expiration
Deployment ID	Used to distinguish installations/deployments	Available as part of LTI registration context	Instance/configuration dependent	Tenant/configuration dependent	Required in every tenant/course context binding

Capability	Canvas	Blackboard Learn	Moodle	D2L Brightspace	Semester requirement
Deep Linking	Configure tool capability/permissions	Must be enabled in admin/tool setup	Version/configuration dependent	Tenant/configuration dependent	Optional but preferred; manual fallback required
NRPS roster service	Available if tool receives appropriate scopes/permissions	Availability depends on admin setup and Learn deployment	Version/configuration dependent	Availability depends on tenant setup	Optional; request only minimum roles/fields
AGS line items/results	Available with granted LTI Advantage scopes	Requires explicit grade-service access configuration	Version/configuration dependent	Service availability/configuration must be tested	Required if Semester writes back grades
Assignment/grade write	LTI Advantage AGS and/or API workflow	LTI grade-service configuration and/or REST workflow	LTI/API approach depends on configuration	LTI/API approach depends on tenant	Preview, human/institution confirmation, idempotency, audit, reconcile
Names/roles claims	Configurable release depending on tool permissions	Admin-controlled user fields/roles	Depends on site configuration	Depends on configuration	Data minimization; no more PII than needed
Content selection	Deep Linking/tool placement configuration	Admin tool placement/content workflow	Version/configuration dependent	Tenant configuration dependent	Optional and must preserve content source/rights
Admin process	Developer Key, tool configuration, deployment	Register tool, approve, configure fields/services	Site admin configures LTI and services	Admin configures tool and entitlement	Per-tenant integration checklist and sandbox validation
Main risk	Scope/key configuration may differ by tenant	Deployment/version and admin-service variance	Site/plugin/hosting variability	Tenant entitlement/service configuration variance	Never assume a service from vendor support statement alone

Canvas’s LTI documentation uses Developer Keys and external-tool configuration; Blackboard configuration requires tool registration, approval, and grade-service access. Moodle’s published LTI 1.3 material describes OAuth 2.0 and OpenID Connect as the basis of its LTI 1.3 security model. [¹⁶³¹][¹⁶²¹]^[1]

LTI 1.3 endpoint discovery record

For every customer integration, collect and validate this exact record in a secure configuration vault:

```
Tenant/customer ID
LMS product, version, hosting model, and region
Issuer
Client ID
Deployment ID
OIDC login initiation URL
Authorization URL
JWKS URL
OAuth access-token URL
Platform public-key rotation behavior
Deep Linking endpoint and scopes
NRPS endpoint and scopes
AGS line-items endpoint and scopes
AGS results/scores endpoint and scopes
```

Permitted user/role claims
 Data-minimization settings
 Admin/technical owner
 Sandbox/test course
 Production course
 Certificate/key rotation owner
 Last successful validation date
 Known constraints and fallback workflow

QTI 3 migration steps and tools

QTI 3 migration needs both a **technical converter** and a **human assurance process**. A valid XML package that changes scoring, accessibility, media, or item intent is not a successful migration.

QTI 3 is designed to exchange item/test content and results among authoring tools, item banks, delivery systems, scoring engines, and learning platforms. ^[1633][1634]

Migration phases

Phase	Activities	Deliverables	Exit gate
0. Govern	Define scope, owners, source/destination systems, rights, security, and acceptance criteria	Migration charter, RACI, data-handling plan	Stakeholders approve what “equivalent” means
1. Preserve	Export all originals, attachments, metadata, media, rubrics, score fixtures, and usage history	Immutable originals, checksums, source inventory	Originals verified and access restricted
2. Inventory	Classify items by source format, interaction type, scoring complexity, accessibility, rights, and risk	Item inventory and migration IDs	Every object is categorized
3. Crosswalk	Map source semantics to QTI 3 constructs; define unsupported-feature treatment	Versioned mapping specification	Scoring/accessibility mapping approved
4. Pilot convert	Convert representative low-, medium-, and high-complexity item samples	Pilot packages, conversion diagnostics	Pilot behavior is accepted
5. Bulk convert	Convert at scale with logs, exception queue, and repeatable pipeline	QTI 3 packages and exception register	No unreviewed errors
6. Validate	Schema, package, scoring, rendering, asset, accessibility, security, and metadata tests	Automated test report	Required pass rate met
7. Destination test	Import into real destination sandbox, deliver sample attempts, score, export results	Destination acceptance report	Behavior matches approved fixtures
8. Round trip	Export from destination, validate again, compare original/migrated/re-exported state	Round-trip evidence	Portability demonstrated
9. Release	Approve banks, assign review dates, archive evidence, and enable use	Release record and support runbook	Content owner signs off

Detailed migration checklist

1. Source inventory

- [] Identify source format: QTI 1.2, QTI 2.1/2.2, proprietary XML/JSON, CSV, Word/PDF, SCORM package, LMS export, or database extract.
- [] Assign a stable migration ID to every bank, test, section, item, asset, rubric, outcome, and external reference.
- [] Capture author, owner, rights/license, review date, course/tenant scope, lifecycle state, language, and use history.

- [] Identify shared stimuli, item dependencies, reusable feedback, question pools, randomization, branching, timing, and attempts.
- [] Record current expected-score fixtures for representative student responses.
- [] Hash/package all source assets and preserve immutable originals.

2. Semantic and scoring crosswalk

- [] Map identifiers and avoid collisions.
- [] Map response declarations, outcome declarations, response processing, template processing, feedback, and scoring logic.
- [] Map partial credit, penalty scoring, negative marking, tolerance, units, rounding, randomized variables, and adaptive branching.
- [] Map test parts, sections, ordering, selection, item pools, presentation rules, timing, attempt policies, and delivery constraints.
- [] Map metadata: outcomes, topic, difficulty, cognitive level, language, author, rights, provenance, review date, and accessibility status.
- [] Document every proprietary construct with one decision:
map exactly / transform / supported extension / manual rebuild /
omit with warning / block migration.

3. Accessibility conversion

- [] Preserve or add alt text, captions, transcripts, and media descriptions.
- [] Verify language metadata and plain instructions.
- [] Ensure keyboard completion of every interaction.
- [] Provide equivalent alternatives for drag-and-drop, hotspot, graph, simulation, audio, video, and color-dependent questions.
- [] Validate accessible math input/rendering.
- [] Test screen-reader labels, focus order, state change announcements, zoom/reflow, contrast, and mobile rendering.
- [] Test timing, attempts, and available accommodation configurations.
- [] Route inaccessible items to a manual-remediation queue.

4. Conversion tools

Use a toolchain rather than a single black-box importer:

Tool category	Purpose	Semester implementation
Source extractor	Pull item banks/assets/metadata from source LMS	Versioned connector; read-only export; source checksums
Canonical inventory store	Maintain migration IDs, metadata, mappings, exceptions	Tenant-scoped migration database with audit events
QTI transformer	Convert source format to QTI 3 package/manifest	Versioned transformation service; deterministic builds
QTI validator	Check schema/package/semantic profile compliance	Automated validation in CI and migration pipeline
Scoring test runner	Execute fixture responses and compare outcomes	Golden-response test suite
Rendering test harness	Render items in target delivery environment	Screenshot/DOM/accessibility snapshot tests
Accessibility test suite	Keyboard, screen reader, contrast, text alternatives	Automated scans plus trained human testing
Asset processor	Package/rewrite media, equations, URLs, fonts	Rights, malware, broken-link, and hash validation
Exception console	Assign, document, approve, and close nonconforming items	Owner, severity, remediation, decision log

Tool category	Purpose	Semester implementation
Acceptance dashboard	Report conversion health and sign-off status	Counts, defects, pass rate, unresolved blockers

5. Validation tests

```
[ ] Validate QTI package manifest and XML/schema structure.
[ ] Validate destination-supported QTI 3 profile.
[ ] Import into a non-production destination bank.
[ ] Test response capture, response processing, feedback, scoring,
    partial credit, randomization, timing, attempts, and branching.
[ ] Compare expected versus actual outcomes using golden response fixtures.
[ ] Test all media/assets; validate rights, rendering, fallback,
    and inaccessible/expired external links.
[ ] Test item metadata, outcome mapping, rubrics, and grade/result reporting.
[ ] Test student/instructor/administrator views.
[ ] Test keyboard, screen reader, zoom/reflow, mobile, and accommodation paths.
[ ] Export the item from destination and independently revalidate the package.
[ ] Record exact version of source, transformer, validator, destination,
    and test fixture used.
```

Tools and validation sources

Use the official QTI 3 specification and implementation guidance as the semantic reference, plus the QTI conformance/certification material where applicable. QTI 3’s information model covers items, sections, and whole tests, while the beginner/implementation guides describe exchange across authoring, item-bank, delivery, scoring, and learning systems. ^{[1633][1634][^1604]}

QTI 3 migration acceptance thresholds

Set thresholds before conversion begins:

```
100% of high-stakes items have human content and accessibility sign-off.
100% of scoring fixtures pass for released items.
100% of external assets are packaged, resolved, or intentionally removed.
0 unresolved critical accessibility barriers.
0 unresolved critical scoring deviations.
100% of exceptions have an owner and disposition.
100% of released items have provenance and version metadata.
100% of final packages successfully import into the target production-equivalent sandbox.
```

AI-integrity tool comparison

There is no defensible universal “accuracy” or false-positive percentage for Gradescope, Copyleaks, or Turnitin. Gradescope is principally a grading-workflow product, while Copyleaks and Turnitin are principally integrity/similarity and AI-writing-signal products; their performance depends heavily on assignment type, language, model/version, threshold, and evaluation dataset.

Criterion	Gradescope	Turnitin	Copyleaks	Semester requirement
Primary workflow	Rubric grading, answer groups, grading assistance	Similarity review and AI-writing signal where licensed	Similarity/AI-writing signal	Source-aware rubric, feedback, grade and review workflow

Criterion	Gradescope	Turnitin	CopyLeaks	Semester requirement
False-positive claim	Not comparable as an AI-writing detector	Require current, independent, scenario-specific evidence	Require current, independent, scenario-specific evidence	Do not accept a single vendor-wide accuracy number
Automated action	Assist grader; human final review	Flag for trained human review only	Flag for trained human review only	Never auto-grade misconduct or impose penalty
Student due process	Institution-configured	Institution-configured	Institution-configured	Evidence view, student explanation, review, appeal, audited decision
Data question	Storage, retention, integration, secondary use	Repository/retention, AI terms, data use	Repository/retention, AI terms, data use	No vendor training on student work by default

Use an institution-controlled, representative evaluation with multiple human reviewers and report false positives, false negatives, precision, recall, confidence intervals, subgroup results, and appeal outcomes—rather than relying on a vendor marketing percentage.

Semester's implementation standard

LTI:

Use as the universal baseline and validate each tenant's actual services.

APIs:

Use only for explicitly approved, documented gaps not covered by LTI.

Rate limits:

Assume variable capacity; adapt from headers and behavior.

Events:

Assume at-least-once, delayed, duplicated, out-of-order, or missing delivery; use durable queues and reconciliation.

Grades:

Write only through previewed, idempotent, audited, reconcilable workflows.

QTI 3:

Convert semantically, validate scoring/accessibility in the destination, and retain the original plus migration evidence.

AI integrity:

Treat tools as signals; preserve human judgment, student notice, response, and appeal.

This gives Semester an integration strategy that works across enterprise LMS environments while remaining more reliable and defensible than a vendor-specific connector.

✱

1. https://developerdocs.instructure.com/services/canvas/external-tools/lti/file.lti_dev_key_config

2. <https://moodledev.io/docs/5.0/apis>

3. <https://oauth.net/2/>

4. https://docs.moodle.org/en/OAuth_2_authentication

5. <https://moodle.org/mod/forum/discuss.php?d=463054>

6. <https://success.vitalsource.com/hc/en-us/articles/31407609441431-Sampling-Insights-Dashboard>
7. <https://support.anthology.com/apex/downloadArticleasPDF?id=kaEPU0000004QVQ2A2>
8. <https://moodle.org/mod/forum/discuss.php?d=357646>
9. <https://developer.blackboard.com/portal/displayApi>
10. <https://epe.brightspotcdn.com/68/c3/8af36fcd4d2d8270166bda5120ee/gwinnett-ga-rostering-solution.pdf>
11. <https://developer.blackboard.com/portal/displayApi/Learn?version=3900.42.0>
12. <https://www.imsglobal.org/spec/qti/v3p0/conf>
13. <https://www.forasoft.com/learn/elearning-video/articles-elearning/integrating-with-moodle>
14. <https://ldbypass.com/lockdown-browser-mac/symptoms/network-keeps-dropping-m4-mac>
15. <https://ldbypass.com/lockdown-browser-mac/symptoms/network-keeps-dropping-intel-mac>
16. <https://studylib.net/doc/17568851/brightspace-platform-10.4.0.-to-10.6.1-known-issues-list>
17. <https://stackoverflow.com/questions/54516476/unable-to-get-authorization-code-on-blackboard-learn>
18. [https://docs.moodle.org/en/Publish as LTI tool](https://docs.moodle.org/en/Publish_as_LTI_tool)
19. <https://learn.microsoft.com/en-us/linkedin/learning/sso-auth/sso-docs/sso-lti-13-blackboard>
20. <https://www.imsglobal.org/spec/qti/v3p0/guide>
21. https://www.imsglobal.org/sites/default/files/spec/qti/v3/info/imsqti_asi_v3p0p1_infomodel_v1p0.html
22. <https://community.d2l.com/brightspace/kb/articles/1191-api-rate-logging-limiting-faq>
23. <https://docs.valence.desire2learn.com/basic/apicall.html>
24. <https://developerdocs.instructure.com/services/canvas/basics/file.throttling>